

Семинар #4: Указатели и структуры. Домашнее задание.

Задача 1. Одна строка

Решения всех подзадач этой задачи – одна строка. Вам нужно, используя любой текстовый редактор, создать файл в формате `.txt` и записать в него ответы на все подзадачи. После этого, файл нужно поместить в ваш репозиторий на github.

1. В следующей программе создайте указатель `p` и инициализируйте его адресом переменной `a`:

```
int main()
{
    int a = 10;
    // Тут нужно написать 1 строку кода
}
```

2. Создайте указатель `p` и инициализируйте его адресом переменной `a`:

```
int main()
{
    float a = 1.1;
    // Тут нужно написать 1 строку кода
}
```

3. Создайте указатель `p` и сделайте так, чтобы он указывал на первый элемент массива (индекс 0):

```
int main()
{
    int array[5] = {10, 20, 30, 40, 50};
    // Тут нужно написать 1 строку кода
}
```

4. Создайте указатель `p` и сделайте так, чтобы он указывал на символ `'A'` из строки `str`:

```
int main()
{
    char str[20] = "Sapere Aude";
    // Тут нужно написать 1 строку кода
}
```

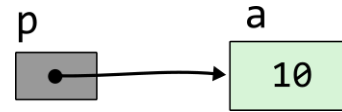
5. Создайте указатель `q` и инициализируйте его адресом переменной `p`:

```
int main()
{
    int a = 10;
    int* p = &a;
    // Тут нужно написать 1 строку кода
}
```

6. В следующей программе удвойте значение переменной `a`, используя только указатель `p`. Можно использовать только указатель `p`. Переменную `a` использовать нельзя.

```
#include <stdio.h>
int main()
{
    int a = 10;
    int* p = &a;
    // Тут нужно написать 1 строку кода

    printf("%i\n", a);
}
```



7. Переведите символ, хранящийся в переменной `a` в верхний регистр, используя только указатель `p`. Можно использовать только указатель `p`. Переменную `a` использовать нельзя.

```
#include <stdio.h>
int main()
{
    char a = 't';
    char* p = &a;
    // Тут нужно написать 1 строку кода

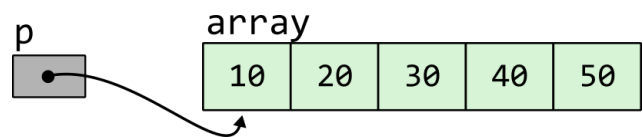
    printf("%c\n", a);
}
```



8. В следующей программе добавьте 1 к четвёртому элементу массива, используя только указатель `p` на первый элемент. Можно использовать только указатель `p`. Массив `a` использовать нельзя.

```
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int* p = &a[0];
    // Тут нужно написать 1 строку кода

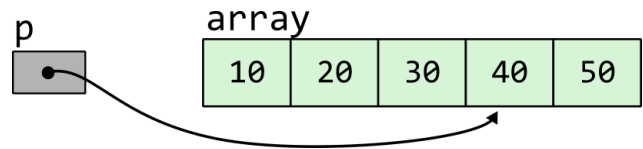
    for (int i = 0; i < 5; ++i)
        printf("%i ", a[i]);
}
```



9. В следующей программе используйте указатель на четвёртый элемент массива(p), чтобы добавить 1 к первому элементу массива. Можно использовать только указатель p. Массив a использовать нельзя.

```
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int* p = &a[3];
    // Тут нужно написать 1 строку кода

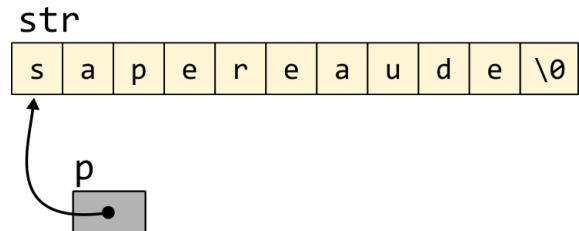
    for (int i = 0; i < 5; ++i)
        printf("%i ", a[i]);
}
```



10. В следующей программе переведите в верхний регистр все буквы строки указатель. Можно использовать только указатель p. Строку str использовать нельзя. Решение – 1 цикл (несколько строк кода).

```
#include <stdio.h>
int main()
{
    char str[] = "sapereaude";
    char* p = &str[0];
    // Тут нужно написать код

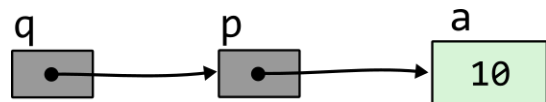
    printf("%s\n", str);
}
```



11. Удвойте значение переменной a, используя указатель q. Можно использовать только указатель q. Использовать переменные a и p нельзя.

```
#include <stdio.h>
int main()
{
    int a = 10;
    int* p = &a;
    int** q = &p;
    // Тут нужно написать 1 строку кода

    printf("%i\n", a);
}
```

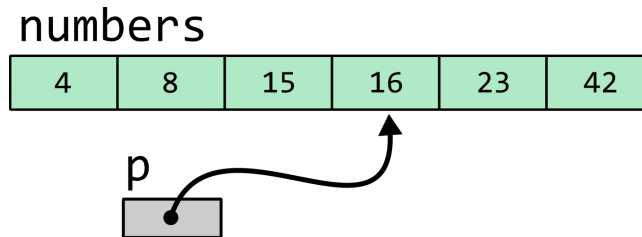


Задача 2. Указатель в массиве

Решения всех подзадач этой задачи – одно число. Вам нужно, используя любой текстовый редактор, создать файл в формате `.txt` и записать в него ответы на все подзадачи. После этого, файл нужно поместить в ваш репозиторий на github.

Пусть есть массив и указатель на 4-й элемент этого массива:

```
int numbers[6] = {4, 8, 15, 16, 23, 42};  
int* p = &numbers[3];
```



Чему равны следующие выражения:

- | | | |
|----------------------------|--------------------------|--|
| 1. <code>numbers[5]</code> | 5. <code>p[0]</code> | 9. <code>*(numbers+5)</code> |
| 2. <code>*p</code> | 6. <code>p[1]</code> | 10. <code>p - numbers</code> |
| 3. <code>*(p+1)</code> | 7. <code>p[-2]</code> | 11. <code>(short*)p - (short*)numbers</code> |
| 4. <code>*(p-2)</code> | 8. <code>*numbers</code> | 12. <code>(char*)p - (char*)numbers</code> |

Задача 3. Куб по указателю

Напишите функцию `cube`, которая будет принимать на вход адрес некоторой переменной типа `float`.

```
void cube(float* px)
```

Функция должна возводить в куб переменную, чей адрес хранит входящий указатель. Вызовите эту функцию из `main` и протестируйте её.

Задача 4. Умножение массива на 2

Напишите функцию

```
void mult2(int* p, size_t n)
```

которая будет принимать указатель на первый элемент некоторого массива `p` и целое число `n`, равное размеру этого массива. Функция должна увеличивать все элементы массива в 2 раза. Решите эту задачу в двух вариантах:

- К указателю можно применять только оператор сложения `+` и оператор разыменования `*`.
- К указателю можно применять только оператор квадратные скобки `[]`.

Задача 5. Квадратное уравнение

Напишите функцию

```
int solve_quadratic(double a, double b, double c, double* px1, double* px2)
```

которая должна решать квадратное уравнение с коэффициентами `a`, `b` и `c`. Результат функция должна записывать по адресам `px1` и `px2`. Функция должна возвращать:

- 0 - если корней нет. По адресам `px1` и `px2` ничего записывать в этом случае не надо.
- 1 - если есть один корень. Его нужно записать по адресу `px1`.
- 2 - если есть два корня. Их нужно записать по адресам `px1` и `px2`.

Все сравнения делать с точностью `eps = 1e-10`.

Задача 6. Обмен

Напишите функцию

```
int exchange(int* pa, int b)
```

которая будет присваивать переменной по адресу `pa` значение `b` и возвращает старое значение переменной, на которую указывает `pa`. Протестируйте функцию с помощью следующего кода:

```
#include <stdio.h>
// Тут нужно написать функцию exchange

int main()
{
    int a = 10;
    printf("%i\n", exchange(&a, 20)); // Напечатает 10
    printf("%i\n", a);                // Напечатает 20
}
```

Задача 7. Максимум по адресу

Напишите функцию

```
int* max(int* pa, int* pb)
```

которая будет принимать два адреса на числа и находить максимум из этих чисел. Функция должна возвращать адрес наибольшего числа. Протестируйте функцию с помощью следующего кода:

```
#include <stdio.h>
// Тут нужно написать функцию max

int main()
{
    int a = 10;
    int b = 30;
    int c = 20;
    *max(max(&a, &b), &c) += 1;
    printf("%i %i %i\n", a, b, c); // Напечатает 10 31 20
}
```

Задача 8. Изменить символы

Напишите функцию

```
void set_characters(char* begin, char* end, char c)
```

которая будет задавать символы в строке символом `c`. Начиная с символа, на который указывает `begin` и заканчивая символом на который указывает `end` (но не включая его). Гарантируется, что `end` указывает на символ, находящийся в этой же строке и не левее символа, на который указывает `begin`.

```
#include <stdio.h>
// Тут нужно написать функцию set_characters

int main()
{
    char s[] = "Sapere Aude";
    set_characters(&s[2], &s[8], 'b');
    printf("%s\n", s); // Напечатает Sabbbbbbbude
    set_characters(s, &s[4], 'a');
    printf("%s\n", s); // Напечатает aaaabbbbude
}
```

Задача 9. Используемые символы

Напишите функцию

```
void used_chars(const char* str, char* used)
```

Которая бы принимала указатель на первый символ строки `str` и указатель на первый символ другой строки `used`. Функция должна находить все буквы в строке `str` и записывать их в `used` в алфавитном порядке. Символы, не являющиеся буквами, игнорируются. Если буква встречается в `str` (заглавная или строчная), в `used` добавляется соответствующая строчная буква. Считайте, что в строке `used` достаточно места.

```
#include <stdio.h>
#include <string.h>
// Тут нужно написать функцию used_chars

int main()
{
    char s[50] = "Sapere Aude";
    char u[30];
    used_chars(s, u);
    printf("%s\n", u); // Напечатает "adeprsu"

    strcpy(s, "123!$@");
    used_chars(s, u);
    printf("%s\n", u); // Ничего не напечатает (только перенос строки)

    strcpy(s, "The Quick Brown Fox Jumps Over The Lazy Dog!");
    used_chars(s, u);
    printf("%s\n", u); // Напечатает "abcdefghijklmnopqrstuvwxyz"
}
```

Задача 10. Максимальная строка

Напишите функцию

```
char* strmax(char** strings, size_t n)
```

которая будет принимать на вход массив строк (на самом деле указатель на первый элемент массива указателей типа `char*`) и размер этого массива. Функция должна будет находить максимальную строку и возвращать указатель на неё. Для сравнения строк используйте функцию `strcmp`. Протестируйте функцию с помощью следующего кода:

```
#include <stdio.h>
#include <string.h>
// Тут нужно написать функцию strmax

int main()
{
    char a[] = "Cat";
    char b[] = "Mouse";
    char c[] = "Wolf";
    char d[] = "Kangaroo";
    char e[] = "Elephant";

    char* animals[5] = {&a[0], &b[0], &c[0], &d[0], &e[0]};
    char* x = strmax(animals, 5);
    printf("%s\n", x); // Напечатает Wolf
}
```

Задача 11. Треугольник

Для описания точек и треугольников на плоскости были определены структуры Point и Triangle:

```
#include <stdio.h>

struct point
{
    double x;
    double y;
};
typedef struct point Point;

struct triangle
{
    Point a;
    Point b;
    Point c;
};
typedef struct triangle Triangle;

// Тут нужно написать все необходимые функции

int main()
{
    Triangle t = {{1.00, 0.00}, {0.50, 2.00}, {0.00, 1.50}};
    printf("Perimeter = %.2f\n", get_triangle_perimeter(&t));

    Point d = {1.0, 1.0};
    print_triangle(&t);
    move_triangle(&t, d);
    print_triangle(&t);
}
```

Напишите следующие функции для работы с этими структурами:

- Функцию `print_point`, которая будет принимать точку и печатать её в формате (1.23, 4.56). То есть в круглых скобках, через запятую и с двумя знаками после запятой.
- Функцию `print_triangle`, которая будет принимать на вход треугольник и печатать координаты треугольника в следующем формате: {(1.00, 0.00), (0.50, 2.00), (0.00, 1.50)}.
- Функцию `distance`, которая будет принимать на вход 2 точки и возвращать расстояние между ними.
- Функцию `get_triangle_perimeter`, которая будет принимать треугольник по константному указателю и возвращать его периметр.
- Функцию `moved_triangle`, которая будет принимать на вход треугольник по константному указателю и одну точку (она будет играть роль вектора-перемещения). Функция должна возвращать новый треугольник, у которого все координаты будут передвинуты на вектор-перемещение.
- Функцию `move_triangle`, которая будет принимать на вход треугольник по указателю и одну точку (она будет играть роль вектора-перемещения). Функция должна менять передаваемый ей треугольник.

Задача 12. Одна строка

Решения всех подзадач этой задачи – одна строка. Вам нужно создать файл в формате `.txt` и, используя любой текстовый редактор, записать в него ответы на все подзадачи. После этого, файл нужно поместить в ваш репозиторий на github.

1. В следующей программе создаётся структура `Book`:

```
struct book
{
    char title[50];
    int pages;
    float price;
};
typedef struct book Book;

int main()
{
    Book b = {"Fahrenheit 451", 400, 700.0};

}
```

- (a) Создайте указатель `pb` и сделайте так, чтобы он указывал на структуру `b`.
- (b) Создайте указатель `pprice` и сделайте так, чтобы он указывал на поле `price` структуры `b`.
- (c) Создайте указатель `pc` и сделайте так, чтобы он указывал символ `'t'` поля `title` структуры `b`.

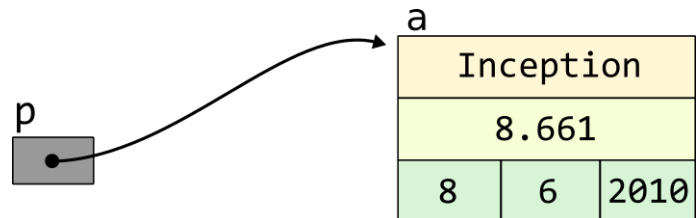
2. В следующей программе была создана структура `a` типа `Movie` и указатель на неё.

```
#include <stdio.h>
struct date
{
    int day;
    int month;
    int year;
};
typedef struct date Date;

struct movie
{
    char title[50];
    float rating;
    Date release_date;
};
typedef struct movie Movie;

int main()
{
    Movie a = {"Inception", 8.7, {8, 6, 2010}};
    Movie* p = &a;

}
```



- (a) Увеличьте на 1 значение поля `rating`, используя только указатель `p`.
- (b) Увеличьте на 1 значение поля месяца выхода фильма, используя только указатель `p`.

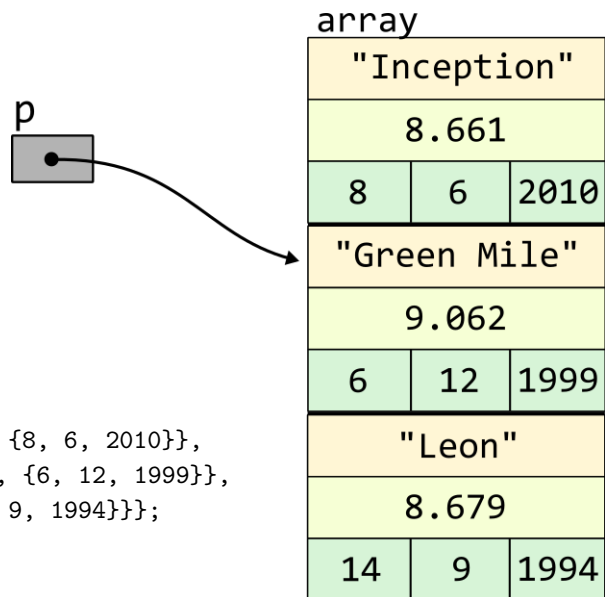
3. В следующей программе был создан массив `array` из структур типа `Movie` и указатель `p`, который указывает на второй элемент массива (`array[1]`).

```
#include <stdio.h>
struct date
{
    int day;
    int month;
    int year;
};
typedef struct date Date;

struct movie
{
    char title[50];
    float rating;
    struct date release_date;
};
typedef struct movie Movie;

int main()
{
    Movie array[3] = {"Inception", 8.661, {8, 6, 2010}},
                  {"Green Mile", 9.062, {6, 12, 1999}},
                  {"Leon", 8.679, {14, 9, 1994}};

    Movie* p = &array[1];
}
```



- (a) Увеличьте на 1 значение рейтинга фильма `Inception`, используя только указатель `p`.
 (b) Удвойте значение года выхода фильма `Leon`, используя только указатель `p`.

Задача 13. Рецензии на компьютерные игры

На вход программе приходит информация о рецензиях компьютерных игр. В первой строке содержится число `n` - количество игр. Далее идут `n` строк. В каждой строке содержится название игры, заканчивающееся двоеточием сразу после идёт целое число `k` - количество оценок, которые эта игра получила, затем идут `k` оценок. Оценка, это число от 1 до 10. Нужно отсортировать все игры по средней оценке и напечатать название игр и их среднюю оценку.

ВХОД	ВЫХОД
5	The Cube, 8.286
Need For Speed: 6 6 1 2 7 5 4	Metal Power, 5.900
Sector: 3 1 4 2	Principle Of Chaos 2, 5.200
The Cube: 7 9 8 7 9 8 10 7	Need For Speed, 4.667
Principle Of Chaos 2: 5 4 3 6 5 7	Sector, 2.333
Metal Power: 10 8 5 3 9 6 2 6 7 5 8	

Для считывания до символа `:` можно использовать `scanf` следующим образом:

```
char title[50];
char temp;
scanf("%[^:]", title); // Считываем название до символа :
scanf("%c", &temp);    // Считываем символ :
...                    // Считываем числа
scanf("%c", &temp);     // Считываем перенос на новую строку ( символ \n )
scanf("%[^:]", title); // Считываем название до символа :
scanf("%c", &temp);     // Считываем символ :
...
```

Протестировать программу можно на файле `videogames.txt`.

Задача 14. Выравнивание типов

Напишите программу, которая будет печатать на экран размеры и выравнивания следующих типов:

```
1. char
2. int
3. size_t
4. int[10]
5. int*
6. struct cat
   {
       char x;
       size_t y;
   };
7. struct dog
   {
       size_t y;
       char x;
   };
8. struct mouse
   {
       char x;
       char y;
       size_t z;
   };
9. struct rat
   {
       char x;
       size_t z;
       char y;
   };
10. struct fox
    {
        char x;
        struct mouse y;
    };

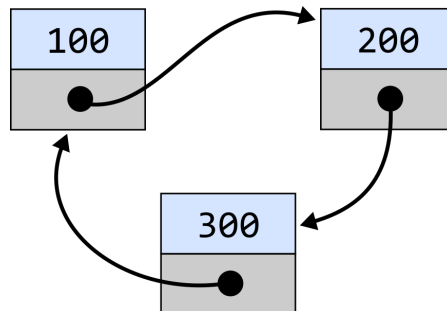
```

Задача 15. Зацикленные структуры

Пусть есть структура `Node`, которая хранит некоторое число и указатель на такую же структуру `Node`:

```
struct node
{
    int value;
    struct node* ptr;
};
typedef struct node Node;
```

Напишите программу, которая будет создавать две такие структуры и инициализировать их поля значениями, показанными на рисунке:



Указатель `ptr` первой структуры должен указывать на вторую структуру, а указатель `ptr` второй структуры, должен указывать на третью структуру, а указатель третьей – на первую.

Напишите программу, которая будет печатать значение `value` структуры, а потом переходить к следующей структуре через поле `ptr`, потом печатать значение `value` уже этой структуры и переходить по полю `ptr` к следующей и так далее до бесконечности.

Необязательные задачи (не входят в ДЗ, никак не учитываются)

Задача 1. Обращенная копия строки

Напишите функцию

```
void reverse_copy(const char* source, char* destination, size_t n)
```

которая принимает указатель на первый элемент некоторой строки (`source`), а также указатель на первый элемент другой строки (`destination`). Функция должна записывать в `destination` обращенную копию строки `source`. Также, на вход функции передаётся `n` – вместимость строки `destination`. Если длина `source` будет больше или равна `n`, то вы должны будете скопировать только `n - 1` символов и поставить нулевой символ в конце.

```
#include <stdio.h>
// Тут нужно написать функции reverse_copy

int main()
{
    char s[10] = "Elephant";
    char d[10];
    reverse_copy(s, d, 10);
    printf("%s\n", d); // Должно напечатать tnahpele

    reverse_copy(s, d, 5);
    printf("%s\n", s); // Должно напечатать tnah
}
```

Задача 2. Печать разных типов:

Напишите функцию `void polyprint(const char* type, void* p)`, которая должна будет печатать то, на что указывает указатель `p`. Тип того, на что указывает `p`, задаётся с помощью первой переменной и может принимать следующие значения:

- Если `type == "Integer"`, то `p` указывает на целое число типа `int`.
- Если `type == "Float"`, то `p` указывает на вещественное число типа `float`.
- Если `type == "Character"`, то `p` указывает на символ (тип `char`).
- Если `type == "String"`, то `p` указывает на первый символ строки.
- Если `type == "IntegerArray 15"`, то `p` указывает на первый элемент массива размером 15. Элементы этого массива имеют тип `int`. Нужно распечатать все элементы через пробел. Тут нужно использовать функцию `sscanf`, для того чтобы распарсить строку `type`.
- В ином случае функция должна печатать `Error!`

В любом случае, в конце функция должна печатать символ перехода на новую строку. Для сравнения строк нужно пользоваться функцией `strcmp`. Протестируйте функцию с помощью следующего кода:

```
#include <stdio.h>

// Тут нужно написать функцию polyprint

int main()
{
    int a = 123;
    polyprint("Integer", &a);
}
```

```

float b = 1.5;
polyprint("Float", &b);
char c = 'T';
polyprint("Character", &c);

char e[] = "Sapere Aude";
polyprint("String", e);
int f[] = {10, 20, 30, 40, 50};
polyprint("IntegerArray 5", f);
}

```

Задача 3. Передача в функцию по указателю

Передача в функцию по значению

```

#include <stdio.h>
struct movie
{
    char title[50];
    float rating;
    struct date release_date;
};
typedef struct movie Movie;

void change_rating(Movie m)
{
    m.rating += 1;
}

int main()
{
    Movie a = {"Inception", 8.661,
               {8, 6, 2010}};
    change_rating(a);
}

```

Память функции main

a

Inception		
8.661		
8	6	2010

Память функции change_rating

m

Inception		
8.661		
8	6	2010

Всё, что передаётся в функцию, копируется (кроме массивов). Поэтому функция `change_rating` будет менять поле `rating` у копии структуры `a`, а изначальная структура не изменится.

Передача в функцию по указателю:

```

#include <stdio.h>
struct movie
{
    char title[50];
    float rating;
    struct date release_date;
};
typedef struct movie Movie;

void change_rating(Movie* pm)
{

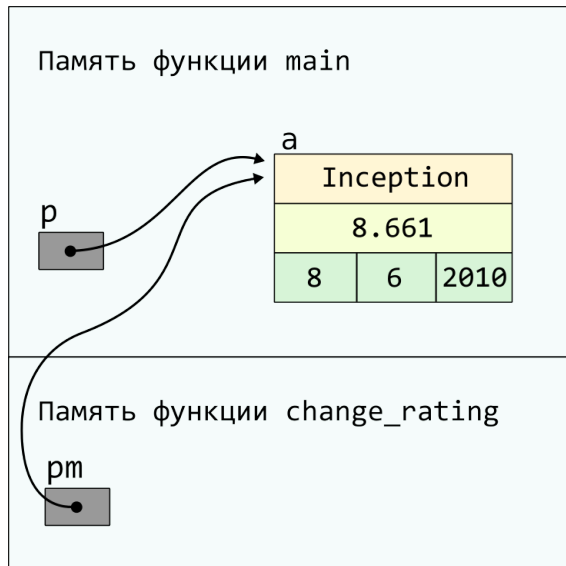
```

```

    pm->rating += 1;
}

int main()
{
    Movie a = {"Inception", 8.661,
               {8, 6, 2010}};
    Movie* p = &a;
    change_rating(&a);
}

```



Теперь в функцию копируется указатель, который содержит адрес структуры **a**. Используя этот указатель, мы можем изменить изначальную структуру. Более того, так как указатель занимает меньше памяти, его копирование в функцию происходит быстрее, чем копирование всей структуры.

Подзадачи:

1. Напишите функцию `void increase_rating(Movie* p)`, которая будет принимать указатель типа `Movie*` и увеличивать рейтинг фильма, на которой указывает `p`, на 1.
2. Напишите функцию `void change_year_of_movies(Movie* p, int size)`, которая принимает на вход указатель на первый элемент массива структур типа `Movie` и размер этого массива. Функция должна увеличивать год выхода всех фильмов на 1. Протестируйте функции, вызвав их из функции `main` с помощью следующего кода:

```
#include <stdio.h>
struct date
{
    int day, month, year;
};
typedef struct date Date;
struct movie
{
    char title[50];
    float rating;
    struct date release_date;
};
typedef struct movie Movie;

void print_date(const Date* pd)
{
    printf("%02d.%02d.%04d", pd->day, pd->month, pd->year);
}

void print_movie(const Movie* pm)
{
    printf("Title: %s\nRating: %.2f\nDate: ", pm->title, pm->rating);
    print_date(&pm->release_date);
    printf("\n");
}
```

```
}  
// Тут вам нужно написать функции increase_rating и change_year_of_movies  
  
int main()  
{  
    Movie a[3] = {"Inception", 8.661, {8, 6, 2010}},  
                 {"Green Mile", 9.062, {6, 12, 1999}},  
                 {"Leon", 8.679, {14, 9, 1994}}};  
    increase_rating()  
    change_year_of_movies(a, 3);  
    for (int i = 0; i < 3; ++i)  
        print_movie(&a[i]);  
}
```